

Akamai SIEM Integration with Splunk Cloud (Victoria)

Complete Operational Reference — Step-by-Step — Fact-Checked

Akamai SIEM Integration with Splunk Cloud (Victoria)

Complete Operational Reference — Step-by-Step — Fact-Checked

Property	Value
Classification	Internal — Security Engineering
Platform	Splunk Cloud Victoria Experience (Dev + Prod)
ES Version	Enterprise Security (ES enabled, RBA not enabled)
Akamai Products	App & API Protector, Kona Site Defender, Bot Manager, Account Protector
Splunkbase TA	Akamai SIEM Integration — App ID 4310
CIM Add-on	Splunk CIM — App ID 1621
Date	February 03, 2026
Version	2.0 — Full Operational Depth

Scope: This document covers the complete end-to-end process: Akamai prerequisites → collector deployment → Splunk Cloud configuration (indexes, HEC tokens, TAs) → field validation → CIM mapping → data model verification → acceptance gates → Prod promotion → failure reference. Every step includes WHAT / WHERE / WHY / HOW / VERIFY with exact Splunk UI paths and SPL queries.

Fact-Checking: Every architectural claim validated against Akamai TechDocs (techdocs.akamai.com) and Splunk documentation (docs.splunk.com). 12 enforced truths govern this document.

Table of Contents

- Section 1** — Executive Summary
- Section 2** — 12 Enforced Architectural Truths
- Section 3** — Architecture: Edge to ES (8-Stage Pipeline)
- Section 4** — Akamai Prerequisites — Step by Step
- Section 5** — Collector Deployment & API Polling
- Section 6** — Splunk Cloud HEC Configuration — Step by Step
- Section 7** — Connecting the Pipeline: Akamai → Collector → HEC
- Section 8** — First Validation: Is Data Arriving?
- Section 9** — Parsing & Field Extraction — Step by Step
- Section 10** — CIM & ES Readiness — Step by Step
- Section 11** — Data Model Validation — Step by Step
- Section 12** — ES Safety: Why CIM Changes Can Break Prod
- Section 13** — Dev → Prod Safety Model
- Section 14** — Acceptance Gates (7 Gates with SPL)
- Section 15** — Failure Modes Reference (14 Modes)
- Section 16** — Prod Promotion Checklist
- Section 17** — Official Documentation References

Section 1 — Executive Summary

This document provides the complete operational procedure for integrating Akamai security events into Splunk Cloud (Victoria Experience) for consumption by Splunk Enterprise Security (ES). It is designed for hands-on security engineers who need exact UI paths, SPL queries, and configuration steps — not just architectural concepts.

The integration follows an 8-stage pipeline: Akamai edge → Security Events Collector (ASEC) → SIEM Integration API → customer-managed connector → Splunk Cloud HEC → index → search-time TA parsing → ES data model consumption. Splunk Cloud does NOT poll the Akamai API. An external connector deployed on customer infrastructure handles all collection.

What This Document Covers

Domain	Coverage
Akamai setup	SIEM toggle, configIDs, API credentials, network path — WHAT/WHERE/HOW/VERIFY
Collector	Deployment options, API polling modes, offset lifecycle, failure handling
Splunk Cloud config	Index creation, HEC token setup, sourcetype selection — exact UI paths
TA installation	Splunkbase download, SSAI install, verification SPL
Field validation	5 sub-steps with SPL queries and decision tables
CIM mapping	TA coverage audit, custom CIM app creation with full code
Data model	5 tstats queries, acceleration health, troubleshooting table
Acceptance gates	7 gates with pass/fail SPL, escalation triggers
Dev → Prod	Safe change protocol, 5 risk scenarios, promotion checklist
Failure reference	14 failure modes (7 API-side, 7 Splunk-side) with fixes

Section 2 — 12 Enforced Architectural Truths

Every claim in this document is governed by these 12 truths. If any content contradicts them, the truth takes precedence.

#	Enforced Truth
1	Akamai provides a generic SIEM Integration API — not Splunk-specific.
2	Akamai provides official SIEM connector packages (Java, CEF) for customer deployment.
3	The SIEM connector runs on customer-managed infrastructure — NOT inside Splunk Cloud.
4	Splunk Cloud does NOT poll the Akamai API. It is a passive HEC receiver only.
5	Splunk Cloud does NOT store Akamai EdgeGrid API credentials.
6	Splunk Cloud is the HEC destination only — it receives pre-collected events.
7	The Akamai Splunk TA provides parsing + partial CIM mapping on the search head.
8	CIM compliance must be validated and potentially extended — never assumed complete.
9	CIM normalization happens at search time via FIELDALIAS, EVAL, and tags.
10	ES does not auto-correlate without enabled correlation searches and proper CIM data.
11	Multiple Akamai products generate events: WAF, DDoS, Bot Manager, Account Protector.
12	Multiple configIds exist — one per security configuration per product/policy/property.

Section 3 — Architecture: Edge to ES (8-Stage Pipeline)

The following 8-stage pipeline describes every component from the Akamai edge to ES. Each stage has a specific owner, location, and responsibility.

Stage	Component	What Happens / Where to Configure
1	Akamai Edge	Akamai CDN evaluates each HTTP request against configured security policies. Requests triggering rules generate security events.
2	Security Events Collector (ASEC)	Akamai-managed storage that holds security events for 12 hours. Events are available via the SIEM Integration API.
3	SIEM Integration API	GET /siem/v1/configs/{configId}. Requires EdgeGrid auth. Returns NDJSON. Rate-limited (429). 2-minute timeout. 600K event cap per request.
4	Customer Connector	Deployed on YOUR infrastructure (Linux VM, Docker, K8s). Polls the API, tracks offsets, handles retries, forwards to HEC. YOU own this.
5	Splunk Cloud HEC	HTTPS POST receiver. Assigns index, sourcetype, source per token config. Created via: Settings → Data Inputs → HTTP Event Collector → New Token .
6	Splunk Cloud Index	Event storage. Created via: Settings → Indexes → New Index . Set retention per compliance. Isolate WAF data from other sources.
7	Search Head + TA	Akamai SIEM Integration add-on (Splunkbase App 4310) installed via SSAI. Provides JSON extraction, FIELDALIAS, EVAL, eventtypes, CIM tags.
8	Enterprise Security	Consumes CIM-normalized data from accelerated data models. Provides correlation searches, notable events, dashboards, risk scoring.



Orange = Akamai-managed | Blue = Customer-managed | Green = Splunk Cloud | Dark Blue = ES

Critical Boundary: The connector (Stage 4) is the ONLY component that talks to the Akamai API. Splunk Cloud receives data passively via HEC. EdgeGrid credentials live ONLY on the connector host — never in Splunk Cloud.

Section 4 — Akamai Prerequisites — Step by Step

Everything in this section happens in the Akamai Control Center (the Akamai web UI). You need a login with sufficient permissions.

Step 4.1 — Verify SIEM Integration Is Enabled

Aspect	Detail
WHAT	SIEM Integration is a per-configuration toggle that enables Akamai to capture security events for that configuration in ASEC. If off, the API returns empty responses.
WHERE	Akamai Control Center → WEB & DATA CENTER SECURITY → Security Configuration → select config → Advanced Settings → Data Collection for SIEM Integrations .
HOW	Toggle must show On . If Off, turn it on and activate to production (Step 4.2). Record the configId displayed on this page.
VERIFY	Toggle is On AND configId is recorded.

Step 4.2 — Verify Configuration Is Activated to Production

Aspect	Detail
WHAT	The security configuration must be active on the production network to generate events.
WHERE	Same config page → Activate tab → check that Production shows 'Active'.
WHY	Configs in staging or draft mode do not produce security events.
VERIFY	Production status shows 'Active' with a recent activation timestamp.

Step 4.3 — Identify All configIds in Scope

Each Akamai security configuration has a unique configId. The SIEM API fetches events per configId. A large enterprise may have multiple security configurations — one per domain, application, or BU.

Action: Ask the Akamai administrator how many security configurations are in scope. Record every configId.

Step 4.4 — Create API Credentials

Aspect	Detail
WHAT	EdgeGrid API credentials that authenticate your collector when calling the SIEM API. Four values: client_token, client_secret, access_token, hostname.
WHERE	Akamai Control Center → ACCOUNT ADMIN → Identity & access → API clients → Create API Client .
HOW	Select SIEM API service → set access to READ-WRITE . Click Create. Akamai shows four values once . Copy all four immediately — client_secret cannot be retrieved again.
WHY	Without these credentials, the collector cannot authenticate. Without READ-WRITE scope, the collector cannot manage offsets.
VERIFY	You have all four values saved in a secure location (vault, secrets manager).

Step 4.5 — Verify API User Role

WHERE: Control Center → Identity & access → Users → select user → Edit roles.

VERIFY: The **Manage SIEM** role is assigned. Without it, API calls return 403 Forbidden.

Step 4.6 — Verify Network Path (Collector → Akamai API)

The collector must reach Akamai API endpoints over HTTPS (port 443). Whitelist:

Domain	Purpose
*.cloudsecurity.akamaiapis.net	Primary SIEM API endpoint
*.luna.akamaiapis.net	Legacy API endpoint (some configs use this)

Critical: If traffic passes through a proxy, the proxy must NOT alter the Host or Authorization headers. EdgeGrid authentication is header-sensitive — any modification causes 401 errors.

Akamai Prerequisites Checklist

✓	Prerequisite	Value / Status
■	SIEM Integration toggle: On	_____
■	Configuration activated to production	_____
■	configId(s) recorded	_____
■	client_token	_____
■	client_secret (stored securely)	_____
■	access_token	_____
■	hostname	_____
■	Manage SIEM role confirmed	_____
■	Network path verified (HTTPS to *.akamaiapis.net)	_____

Section 5 — Collector Deployment & API Polling

The collector is the bridge between Akamai and Splunk. It runs on YOUR infrastructure. The Akamai TA on the Splunk search head does include a Java-based modular input, but we do NOT use it because Splunk Cloud does not allow OS-level processes, credential storage is limited, and troubleshooting is restricted.

Collector Responsibilities

#	Responsibility	Detail
1	Poll the API	Call SIEM API every 30-60 seconds. Pass configId(s) and EdgeGrid credentials.
2	Track checkpoints	Store the offset from each response. Pass in next request. Prevents duplicates.
3	Handle failures	Retry for 429, timeouts, network errors. Use time-based replay for >12h downtime.
4	Forward to Splunk	POST each event as JSON to Splunk Cloud HEC with correct index, sourcetype, token.

API Polling Modes

Mode	How It Works	When to Use
Offset-based (recommended)	Each response contains a ResponseContext with an offset string. Store offset, pass in next request. API returns only new events.	Steady-state collection.
Time-based	Pass 'from' and 'to' epoch timestamps. API returns all events in range.	Catching up after downtime. Replaying a time window. Initial backfill.

Critical API Behaviors

Behavior	What Your Collector Must Do
NDJSON format	Each line except last is one event. Last line = ResponseContext. Do NOT send it to Splunk.
5-second latency	Built-in delay. Offset mode recovers next call. Do not compensate.
600K event cap	Default limit=10,000. If 'limit':true in ResponseContext, increase limit or poll faster.
2-minute timeout	Truncated response has no ResponseContext. Re-call with same offset, lower limit.
HTTP 429 — Rate limit	No X-RateLimit-Reset header. Exponential backoff: 30s → 60s → 120s.
Base64-encoded fields	ruleActions, ruleData, ruleMessages, ruleSelectors, ruleTags are base64. Decode in collector or at search time.
12-hour replay	If offline, replay from within past 12 hours using time-based mode.
Offset persistence	If offset lost, fall back to time-based mode. Accept possible duplicates.

Section 6 — Splunk Cloud HEC Configuration — Step by Step

HEC (HTTP Event Collector) is Splunk's built-in HTTPS endpoint for receiving data. In Splunk Cloud, HEC is the primary supported ingestion method for external data.

Step 6.1 — Create a Dedicated Index

Aspect	Detail
WHAT	A new index specifically for Akamai security events. Isolating WAF data makes searching faster, retention management easier, and access control clearer.
WHERE	Splunk Cloud → Settings → Indexes → New Index .
HOW	Choose a descriptive name. Set retention per compliance. Leave other settings at defaults.
WHY	If WAF events land in 'main' or a shared index, they become difficult to manage, hard to find, and impossible to set separate retention.
VERIFY	Run: <code> eventcount index=<your_index></code> — confirms index exists (count will be 0 initially).

Record your value: Index name = `<your_index>` = _____

Step 6.2 — Choose a Sourcetype

The sourcetype is the label Splunk uses to determine how to parse the data. If you plan to install the Akamai TA, discover what sourcetype it expects first (typically **akamai:siem**) and use that. If not using a TA, choose any descriptive name.

Record your value: Sourcetype = `<your_sourcetype>` = _____

Step 6.3 — Create the HEC Token

Aspect	Detail
WHAT	An authentication key the collector includes in every POST request to prove it is authorized to send data to this Splunk Cloud instance.
WHERE	Splunk Cloud → Settings → Data Inputs → HTTP Event Collector → New Token .
HOW	Name: descriptive (e.g., 'Akamai WAF Collector'). Source type: set to <code><your_sourcetype></code> . Allowed indexes: select <code><your_index></code> . Default index: <code><your_index></code> . Click Submit. Record the token.
WHY	The token controls what the collector can write to. Restricting to a single index and sourcetype prevents accidental data landing in wrong locations.
VERIFY	The token appears in the HEC list with status 'Enabled'.

Step 6.4 — Record the HEC Endpoint URL

Your Splunk Cloud HEC endpoint URL follows this pattern:

```
https://http-inputs-<your-stack>.splunkcloud.com:443/services/collector
```

Check your Splunk Cloud admin panel for the exact URL. Record it — the collector needs this.

Step 6.5 — Test HEC Independently

Before connecting the collector, verify HEC works by sending a test event:

```
curl -k "https://<HEC_ENDPOINT>/services/collector/event" \ -H "Authorization: Splunk <HEC_TOKEN>" \ -d '{"event":"HEC test","sourcetype":"<your_sourcetype>","index":"<your_index>"}
```

Expected response: {"text":"Success", "code":0}

Verify in Splunk:

```
index=<your_index> "HEC test" earliest=-5m
```

If you see the test event, HEC is working. If not, check: token enabled? Correct URL? Firewall allows HTTPS to Cloud? Index name spelled correctly?

Section 7 — Connecting the Pipeline: Akamai → Collector → HEC

At this point you have: Akamai API credentials (Section 4), a collector host (Section 5), and a working HEC endpoint (Section 6). This section connects them.

Configuration That Lives on the Collector

Setting	Value	Where It Came From
Akamai client_token	(from Step 4.4)	Akamai Control Center
Akamai client_secret	(from Step 4.4)	Akamai Control Center
Akamai access_token	(from Step 4.4)	Akamai Control Center
Akamai hostname	(from Step 4.4)	Akamai Control Center
configId(s)	(from Step 4.3)	Akamai Control Center
HEC endpoint URL	(from Step 6.4)	Splunk Cloud admin panel
HEC token	(from Step 6.3)	Splunk Cloud HEC config
Target index	<your_index>	Step 6.1
Target sourcetype	<your_sourcetype>	Step 6.2

HEC Payload Format

The collector should POST each Akamai event to HEC as a JSON envelope:

```
{ "index": "<your_index>", "sourcetype": "<your_sourcetype>", "time": <epoch_timestamp>, "event":
{ ... the Akamai security event JSON ... } }
```

Timestamp: Extract the epoch timestamp from the Akamai event and pass it as 'time'. If omitted, Splunk uses receive time, creating detection lag.

Start the Collector — What to Watch

Signal	Detail
■ Good signs	Successful API calls, events returned, successful HEC POSTs, offset advancing.
■ Bad signs	HTTP 401/403 from Akamai (credentials — Step 4.4). HTTP 403 from HEC (token — Step 6.3). Empty responses (SIEM not enabled — Step 4.1). Timeouts (network — Step 4.6).

Section 8 — First Validation: Is Data Arriving?

Before worrying about parsing, CIM, or data models, answer one question: **are events landing in Splunk?**

Step 8.1 — Discover What Arrived

```
| tstats count WHERE index=* earliest=-24h by index | sort -count | head 20
```

Lists all indexes with recent events. Look for yours.

```
index=<your_index> earliest=-24h | stats count by sourcetype source | sort -count
```

Shows all sourcetype/source combinations in your index. Record them.

Step 8.2 — Confirm Event Flow

```
index=<your_index> sourcetype=<your_sourcetype> earliest=-30m | stats count | eval  
status=if(count>0, "PASS - ".count." events in last 30 minutes", "FAIL - no events. Check  
collector logs, HEC token, network path.")
```

Step 8.3 — Inspect Raw Events

```
index=<your_index> sourcetype=<your_sourcetype> earliest=-1h | head 1 | table _time host source  
sourcetype _raw
```

If _raw looks like...	Format	Meaning
{"type":"akamai_siem","attackData":{...}}	JSON	Expected. TA should handle parsing.
CEF:0 Akamai ...	CEF	Different connector. Different parsing rules needed.
key=value pairs or syslog wrapper	Structured text	Intermediary reformatted. Custom parsing needed.
Binary / garbled / base64 blob	Problem	Collector output is broken. Fix before proceeding.

Step 8.4 — Check for Continuous Flow

```
index=<your_index> sourcetype=<your_sourcetype> earliest=-24h | timechart span=1h count AS  
event_count
```

Every hour should have events. Gaps = collector outage, credential expiry, or API issues.

Section 9 — Parsing & Field Extraction — Step by Step

Events are in Splunk. Now verify that Splunk can extract meaningful fields.

Where Parsing Happens in Splunk Cloud

Location	What Happens	Who Controls It
Collector (before Splunk)	Your collector transforms JSON — flattening, decoding, renaming — before POSTing to HEC.	You. Full control.
Customer HF (optional)	Heavy Forwarder applies props.conf/transforms.conf at index time.	You. Full control of HF configs.
Search Head (search-time)	TA installed on SH applies EXTRACT, REPORT, FIELDALIAS, EVAL when you search. Fields are computed dynamically.	You install TA via SSAI. TA controls rules.

Most common pattern: Events arrive as raw JSON → TA on SH provides search-time extraction → fields appear when you search.

Step 9.1 — Field Population Check

```
index=<your_index> sourcetype=<your_sourcetype> earliest=-1h | head 100 | fieldsummary | where
count > 0 AND NOT match(field, "^(date_|_|punct|timeendpos|timestartpos)") | eval
pct=round(count/100*100, 1)."% | sort -count | table field count distinct_count pct
```

If You See...	It Means...	Action
Many fields: attackData.clientIP, httpMessage.host, etc.	JSON extraction or TA is working.	Proceed to Step 9.2.
Only metadata: _raw, _time, host, source, sourcetype	No field extraction happening.	Install a TA (Step 9.3) or fix collector output.
CIM-style fields: src, dest, action	TA is installed AND providing CIM mapping.	Jump to Section 10.

Step 9.2 — Critical Field Verification

```
index=<your_index> sourcetype=<your_sourcetype> earliest=-1h | head 50 | eval
has_client_ip=if(isnotnull(coalesce('attackData.clientIP','src')),1,0) | eval
has_method=if(isnotnull(coalesce('httpMessage.method','http_method')),1,0) | eval
has_host=if(isnotnull(coalesce('httpMessage.host','dest')),1,0) | eval
has_rule=if(isnotnull(coalesce('attackData.ruleActions','action')),1,0) | stats
avg(has_client_ip) AS client_ip, avg(has_method) AS method, avg(has_host) AS host, avg(has_rule)
AS rule_action | foreach * [eval <<FIELD>>=round(<<FIELD>>*100,0)."%"]
```

Pass: Client IP, method, host, and rule action > 90%. **Fail:** Any critical field at 0% — fix before proceeding.

Step 9.3 — Check If a TA Is Installed on the Search Head

```
| rest /services/apps/local splunk_server=local | search label="*kamai*" OR label="*SIEM*" OR
title="*kamai*" | table title label version updated
```

If no results: no Akamai TA is installed. If fields are also missing (Step 9.1), installing a TA is the most likely fix.

Step 9.4 — If Fields Are Missing: Diagnosis and Fix

#	Cause	How to Check	Fix
1	No TA on SH	Step 9.3 returns empty.	Install vendor TA via SSAI.
2	TA installed, sourcetype mismatch	rest /services/configs/conf-props search title="**kamai*" — compare with your sourcetype.	Change sourcetype to match TA, or create custom stanza.
3	Events are not JSON	Step 8.3 showed CEF or other format.	Change collector to output JSON.
4	Nested JSON not extracted	Top-level OK, attackData.* missing.	Set KV_MODE=json via custom app, or flatten in collector.
5	Non-standard field names	Fields exist with unexpected names.	Create FIELDALIAS entries in a custom app.

Step 9.5 — Verify Active Parsing Configuration (Cloud-Safe)

In Splunk Cloud you cannot use btool. Use the REST API instead:

```
| rest /services/configs/conf-props/<your_sourcetype> splunk_server=local | transpose | rename column AS setting, "row 1" AS value
```

Look for EXTRACT-, REPORT-, FIELDALIAS-, EVAL-, LOOKUP-, and KV_MODE entries. If none exist, the sourcetype has no extraction rules.

Section 10 — CIM & ES Readiness — Step by Step

The Common Information Model (CIM) is a set of standard field names that Splunk ES uses across all data sources. Without CIM mapping, ES cannot find your events — its dashboards, correlation searches, and threat intelligence all query CIM field names, not vendor-specific ones.

CIM mapping does NOT happen automatically. It requires either a TA that defines FIELDALIAS/EVAL rules for your sourcetype, or manual configuration.

What the Akamai TA Provides (Splunkbase App 4310)

What the TA Provides	Detail
JSON field extraction	KV_MODE=json for akamai:siem sourcetype. Extracts nested fields.
CIM field aliases	FIELDALIAS definitions: attackData.clientIP → src, etc.
Eventtypes and tags	Defines eventtypes (e.g., akamai_siem_attack) and tags (web, attack) for data model inclusion.
Lookup definitions	Value normalization lookups (e.g., action code mapping).

What the TA Does NOT Guarantee: (1) All CIM fields populated — some may be empty if the underlying vendor field is absent. (2) Sourcetype match — TA targets 'akamai:siem'; different sourcetype = rules never fire. (3) Complete tag assignment. (4) Data model population. (5) Missing fields like vendor_product, app.

Step 10.1 — Check If CIM Fields Exist

```
index=<your_index> sourcetype=<your_sourcetype> earliest=-1h | head 100 | fieldsummary | search
field IN ("src","dest","action","http_method","url","status",
"http_user_agent","vendor_product","app","bytes") | eval pct=round(count/100*100,1)."% | table
field count pct
```

Step 10.2 — Required CIM Fields for WAF Events

CIM Field	Maps From (Vendor)	Data Model	Required?
src	attackData.clientIP	Web	Yes — attacker IP
dest	httpMessage.host	Web	Yes — target hostname
http_method	httpMessage.method	Web	Yes
url	httpMessage.path	Web	Yes — requested URI
status	httpMessage.status	Web	Recommended
action	attackData.action (normalized)	Web	Yes — allowed/blocked
http_user_agent	httpMessage.requestHeaders.User-Agent	Web	Recommended
vendor_product	(static: 'Akamai WAF')	Web	Recommended
app	(static: 'Akamai App & API Protector')	Web	Recommended

Step 10.3 — Remediation Options for Missing CIM Fields

#	Option	When to Use
1	Install/update vendor TA on SH via SSAI	Most reliable. TA includes FIELDALIAS rules.
2	Fix sourcetype mismatch	TA is installed but sourcetype doesn't match TA stanza.
3	Create custom CIM mapping app on SH	TA installed but some fields still missing. Or no vendor TA exists.

Step 10.4 — Creating a Custom CIM Mapping App

When the TA doesn't cover all needed CIM fields, create a minimal Splunk app:

App name: SA-akamai-cim-overrides

App structure:

```
SA-akamai-cim-overrides/ ■■■ default/ ■ ■■■ app.conf ■ ■■■ props.conf ← CIM field mappings go
here ■ ■■■ eventtypes.conf ← (if needed) ■ ■■■ tags.conf ← (if needed) ■■■ metadata/ ■■■
default.meta
```

props.conf — CIM Mapping Examples

```
[<your_sourcetype>] # CIM FIELD ALIASES – rename vendor field to CIM field FIELDALIAS-akamai_src =
attackData.clientIP AS src FIELDALIAS-akamai_dest = httpMessage.host AS dest FIELDALIAS-akamai_url
= httpMessage.path AS url FIELDALIAS-akamai_method = httpMessage.method AS http_method
FIELDALIAS-akamai_status = httpMessage.status AS status FIELDALIAS-akamai_ua =
httpMessage.requestHeaders.User-Agent AS http_user_agent # CIM EVAL FIELDS – transform values to
CIM standard EVAL-action = case( \ match(attackData.action, "(?i)^deny$"), "blocked", \
match(attackData.action, "(?i)^alert$"), "allowed", \ match(attackData.action, "(?i)^monitor$"),
"allowed", \ match(attackData.action, "(?i)^mitigated$"), "blocked", \ true(), attackData.action)
# Static vendor identification EVAL-vendor_product = "Akamai WAF" EVAL-app = "Akamai App & API
Protector"
```

Deployment Steps

#	Step	Detail
1	Package as .tar.gz or .spl	tar -czf SA-akamai-cim-overrides.tar.gz SA-akamai-cim-overrides/
2	Deploy to SH via SSAI	Splunk Cloud → Apps → Install App from File (SSAI) or support ticket.
3	Validate immediately	Re-run Step 10.1 CIM field check. All targeted fields should populate.
4	Document	Record which CIM fields are from the vendor TA vs your custom app. Matters during TA upgrades.

Rules: (1) FIELDALIAS only works when source field exists. (2) EVAL overrides FIELDALIAS if both map to same CIM field. (3) These rules layer ON TOP of vendor TA — they don't replace it. (4) **Never edit vendor TA files directly** — SSAI updates overwrite your changes.

Section 11 — Data Model Validation — Step by Step

ES correlation searches query data in two ways: (1) directly against the index, or (2) against accelerated data models using | tstats. Data models are faster but require proper CIM tagging and acceleration.

Step 11.1 — Discover Available Data Models

```
| rest /services/datamodel/model splunk_server=local | table title description acceleration
eai:acl.app | search title="*Web*" OR title="*Intrusion*" OR title="*Network"
```

Record the exact title. **Your value:** <candidate_dm> = _____

Step 11.2 — Check If Events Populate the Data Model

Step 11.2a — Generic check

```
| tstats count FROM datamodel=<candidate_dm> WHERE (index=<your_index>) earliest=-4h by sourcetype
| sort -count
```

Step 11.2b — Akamai-specific: Web data model

```
| tstats count FROM datamodel=Web WHERE (index=<your_index> sourcetype=<your_sourcetype>)
earliest=-4h BY sourcetype
```

Step 11.2c — Inspect CIM fields inside the data model

```
| tstats count FROM datamodel=Web WHERE (index=<your_index> sourcetype=<your_sourcetype>)
earliest=-1h BY Web.src, Web.dest, Web.url, Web.http_method, Web.status, Web.action | head 20
```

Step 11.2d — Check Intrusion_Detection data model

```
| tstats count FROM datamodel=Intrusion_Detection WHERE (index=<your_index>
sourcetype=<your_sourcetype>) earliest=-4h BY sourcetype
```

Step 11.2e — Full data model content search (debugging)

```
| datamodel Web search | search index=<your_index> sourcetype=<your_sourcetype> | head 10 | table
_time, src, dest, url, http_method, status, action
```

Interpreting Results

Result	Meaning	Action
Your sourcetype, count > 0	■ Events are in the data model.	Check acceleration (Step 11.3).
11.2b returns results, 11.2c shows nulls	Events in model but CIM fields incomplete.	Fix CIM mapping (Step 10.4). Re-run.
11.2b empty, 11.2e returns results	Acceleration stale or off.	Enable/rebuild acceleration (Step 11.3).
Both empty	Events NOT in data model at all.	See troubleshooting below.

Why Events May Not Appear in a Data Model

#	Cause	Check	Fix
1	CIM tags not set	index=... head 1 table tag::*	TA must define eventtypes.conf + tags.conf.

#	Cause	Check	Fix
2	Index not in allow list	Apps → Manage Apps → Splunk CIM → Set Up → check index constraint.	Add your index to the data model's allowed indexes.
3	Acceleration not enabled	Settings → Data Models → check acceleration status.	Enable acceleration for the target data model.
4	Acceleration still building	Settings → Data Models → expand → check completion %.	Wait 5-15 minutes for new data to appear.
5	Tag allow list restrictive	CIM Setup page → Tag Allow List.	Add required tags.

Step 11.3 — Verify Acceleration Health

```
| rest /services/admin/summarization splunk_server=local | search title="*<candidate_dm>*" | table
title summary.complete summary.is_inprogress summary.earliest_time summary.latest_time
```

Pass: summary.complete close to 1.0 and summary.latest_time within the last hour.

Section 12 — ES Safety: Why CIM Changes Can Break Prod

CIM Is Shared Infrastructure. ES does not maintain separate CIM configurations per data source. When you add a FIELDALIAS, EVAL, eventtype, or tag for Akamai, those rules operate at the sourcetype level across the entire search head. If incorrect, they affect EVERY data source sharing the same data model.

How Akamai CIM Changes Can Break ES

#	Risk	What Happens	Prevention
1	Incorrect FIELDALIAS	Wrong src value → false positives/negatives in correlation searches.	Validate every alias in Dev with real data before Prod.
2	Overly broad tags	Loose eventtype constraint pulls non-WAF events into Web data model.	Scope eventtypes narrowly. Test with datamodel Web search stats count by sourcetype.
3	Conflicting EVAL overrides	EVAL-action for Akamai conflicts with another TA's EVAL-action.	Always scope to specific [sourcetype] stanza. Never use [default].
4	Data model acceleration corruption	Tag changes make acceleration summaries stale.	Rebuild acceleration after any tag change. Verify with tstats.
5	Editing vendor TA directly	Changes overwritten silently when TA is updated via SSAI.	Never edit vendor TA files. All custom mappings in SA-akamai-cim-overrides.

The Safe Change Protocol

Step	Action	Detail
1	Make the change in Dev only	Add FIELDALIAS, EVAL, tag to custom CIM app. Deploy via SSAI to Dev SH.
2	Run CIM validation (Step 10.1)	Confirm target fields populate. Confirm non-target fields unaffected.
3	Check data model impact	datamodel Web search stats count by sourcetype — only expected sources.
4	Rebuild acceleration if needed	If tags changed, disable/re-enable acceleration or wait for rebuild.
5	Run acceptance gates (Section 14)	All gates must pass in Dev before proceeding.
6	Deploy identical change to Prod	Same app version, same SSAI process. Run gates again in Prod.

Section 13 — Dev → Prod Safety Model

The client operates separate Dev and Prod Splunk Cloud environments. Production must never be used for experimentation. Every change must be validated in Dev first.

Dev vs Prod Responsibilities

Step	Dev	Prod
1	Enable SIEM Integration per configId	Same configId(s)
2	Create Dev index, HEC token, sourcetype	Create Prod index, HEC token (identical settings)
3	Install Akamai TA via SSAI on Dev SH	Install same TA version via SSAI on Prod SH
4	Deploy collector pointing to Dev HEC	Clone collector config → Prod HEC endpoint + token
5	Validate CIM field mapping (Step 10.1)	Confirm CIM fields match Dev results
6	Create/test custom CIM app if needed	Deploy identical custom CIM app via SSAI
7	Validate data model + acceleration	Confirm data model population matches Dev
8	Test correlation searches	Enable validated detections

Steps 1-7 happen entirely in Dev. Step 8 is the only step that touches Prod. If you are tempted to skip Dev validation 'because it's just one field alias' — do not. A single incorrect CIM alias in Prod can break ES correlation searches across ALL data sources sharing the same data model.

Section 14 — Acceptance Gates (7 Gates with SPL)

Run every gate below. **ALL must pass** before declaring onboarding complete.

Gate 1 — Ingestion: Events Arriving Continuously

```
index=<your_index> sourcetype=<your_sourcetype> earliest=-24h | timechart span=1h count AS event_count | eval gate=if(event_count>0, "PASS", "FAIL - gap in this hour")
```

PASS: No hour with zero events in 24h. **FAIL:** Gaps = collector outage.

Gate 2 — Ingestion: Lag Is Acceptable

```
index=<your_index> sourcetype=<your_sourcetype> earliest=-4h | eval lag=_indextime - _time | stats p50(lag) AS p50, p90(lag) AS p90, max(lag) AS max_lag | eval gate=if(p90 < 600, "PASS", "FAIL - p90 lag > 10 minutes")
```

Gate 3 — Ingestion: No Significant Duplicates

```
index=<your_index> sourcetype=<your_sourcetype> earliest=-24h | stats count by _raw | where count > 1 | stats count AS dup_groups | eval gate=if(dup_groups < 10, "PASS", "REVIEW - ".dup_groups." duplicate groups")
```

Gate 4 — Parsing: Critical Fields Extractable

Re-run the Step 9.2 query. Client IP, method, host, and rule action must be > 90%.

Gate 5 — CIM: Core Fields Populated

Re-run the Step 10.1 query. src and action must be > 90%.

Gate 6 — Data Model (If Applicable): Events in tstats

Re-run the Step 11.2 query. Count must be > 0.

Gate 7 — Hygiene: No Unexpected Sourcetypes

```
index=<your_index> earliest=-24h | stats count by sourcetype
```

Only expected sourcetype(s) should appear.

Gate Summary

Gate	Domain	Status	If FAIL
1	Continuous flow	PASS / FAIL	Check collector logs → Section 7
2	Ingestion lag	PASS / FAIL	Increase collector poll rate → Section 5

Gate	Domain	Status	If FAIL
3	Duplicates	PASS / REVIEW	Check offset persistence → Section 5
4	Field extraction	PASS / FAIL	Fix parsing → Section 9
5	CIM mapping	PASS / FAIL	Install/fix TA → Section 10
6	Data model	PASS / N/A	Fix tags/indexes → Section 11
7	Index hygiene	PASS / REVIEW	Fix HEC/collector config → Section 7

Escalation Triggers

Condition	Evidence to Capture	Escalation Target
Zero events > 2 consecutive hours	Collector logs, last offset, API response codes	Platform Eng + Akamai account team
p90 lag > 30 min and growing	Collector throughput, API response sizes	Platform Eng (collector capacity)
Critical fields at 0% despite TA	Sample _raw, fieldsummary, REST conf-props	Detection Eng (parsing fix)
Akamai API returning 403	Full HTTP response body	Akamai account team
Akamai API returning 429 frequently	Request rate logs, number of collectors	Akamai account team + Platform Eng

Section 15 — Failure Modes Reference (14 Modes)

API-Side Failures

#	Failure	Symptoms	Fix
F1	429 rate limit	Collector gets 429. No reset header.	Exponential backoff. Reduce poll frequency. Check for competing collectors.
F2	Expired credentials	401/403 from API.	Re-provision in Control Center. Update collector.
F3	Offset lost	Duplicate spike after restart.	Time-based replay. Dedup at search time.
F4	Offline > 12h	Unrecoverable gap possible.	Document the gap. Time-based replay for events within 12h. Contact Akamai support.
F5	2-min timeout	Incomplete response, no ResponseContext.	Lower limit parameter. Re-call with same offset.
F6	SIEM not enabled	Empty API responses.	Toggle On in Control Center (Step 4.1).
F7	Proxy interference	Cannot reach API or garbled responses.	Whitelist *.akamaiapis.net. Preserve headers.

Splunk-Side Failures

#	Failure	Symptoms	Fix
F8	HEC token disabled	403 from HEC. No events.	Re-enable in Settings → Data Inputs → HEC .
F9	Wrong index	Events in main or unexpected index.	Check HEC token's index settings.
F10	Sourcetype mismatch	TA rules do not fire.	Match sourcetype to TA, or create custom stanza.
F11	TA not on SH	No vendor fields extracted.	Install via SSAI. Apps → Install App from File .
F12	Nested JSON	Top-level OK, nested missing.	KV_MODE=json via custom app or flatten in collector.
F13	Base64 not decoded	Rule fields show encoded strings.	Decode in collector or search-time EVAL.
F14	DM acceleration stale	tstats returns 0 but datamodel search returns events.	Rebuild acceleration: Settings → Data Models → disable/enable .

Section 16 — Prod Promotion Checklist

Dev is validated. Replicate the identical configuration to Production. **No changes, no experiments — only replication of what passed gates in Dev.**

✓	Prod Promotion Task	Detail
■	Deploy collector to Prod infrastructure	Clone Dev config. Change ONLY: HEC URL → Prod, HEC token → Prod, index → Prod index. Same EdgeGrid credentials, same configlds, same polling mode.
■	Create Prod index and HEC token	Identical settings to Dev. Create via Splunk Cloud Prod UI: Settings → Indexes → New Index , then Settings → Data Inputs → HEC → New Token .
■	Install Akamai TA on Prod search head	Same version as Dev. Via SSAI: Apps → Install App from File . Confirm installation.
■	Deploy custom CIM app (if used)	Deploy SA-akamai-cim-overrides to Prod SH via SSAI — same version as Dev.
■	Run all acceptance gates in Prod	Re-run every gate from Section 14 against Prod data. All must pass.
■	Validate Prod data model population	tstats count FROM datamodel=Web WHERE (index=<prod_index>) BY sourcetype — must match Dev.
■	Enable detections in Prod	Only after all Prod gates pass. Enable correlation searches targeting Akamai events.

What Should NOT Be Done Yet

Action	Why Not
Do not enable RBA risk scoring	Client is not using RBA. Adding scores before framework is operational creates noise.
Do not build complex multi-source correlations	Until other sources (VPN, endpoint, identity) are also onboarded and CIM-mapped, cross-source detection won't work.
Do not tune detection thresholds	Let data flow 1-2 weeks to establish baselines first.

Section 17 — Official Documentation References

These are the authoritative sources for every claim in this document.

Resource	URL
Akamai SIEM Integration API	techdocs.akamai.com/siem-integration/reference/api
Akamai SIEM Connector Guide	techdocs.akamai.com/siem-integration/docs/overview
Akamai Control Center	control.akamai.com
Akamai Splunk TA (Splunkbase)	splunkbase.splunk.com/app/4310
Splunk CIM Add-on (Splunkbase)	splunkbase.splunk.com/app/1621
Splunk CIM Manual	docs.splunk.com/Documentation/CIM
Splunk ES Data Models	docs.splunk.com/Documentation/ES
Splunk Cloud HEC Docs	docs.splunk.com/Documentation/SplunkCloud/latest/Data/UsetheHTTPEventCollector
Splunk Cloud SSAI	docs.splunk.com/Documentation/SplunkCloud/latest/Admin/SelfServiceAppInstall

Quick Facts Reference

Property	Value
API Endpoint	GET /siem/v1/configs/{configId}
Authentication	Akamai OPEN EdgeGrid (4 values)
Event Retention	12 hours in ASEC
Max Events/Request	600,000 (default 10,000)
Response Format	NDJSON (last line = ResponseContext)
Base64 Fields	ruleActions, ruleData, ruleMessages, ruleSelectors, ruleTags
Rate Limiting	HTTP 429, no reset header
Inherent Latency	5 seconds (offset mode recovers)
Response Timeout	2 minutes
Security Products	App & API Protector, Kona Site Defender, WAP, Client Reputation, Bot Manager, Account Protector
Splunkbase TA	App ID 4310 — SH only — parsing + CIM only — NOT for collection